

第3章

形式语言与自动机



3.1 基本概念



3.1 基本概念

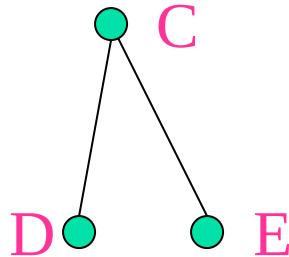
□树 (tree):

- 树是一种数据结构。
- 一个连通的无回路的无向图称为树(或称自由树)。
- 如果树中有一个结点被特别地标记，则这棵树被称之为根树，这个被特别标记的 结点被称之为根结点。

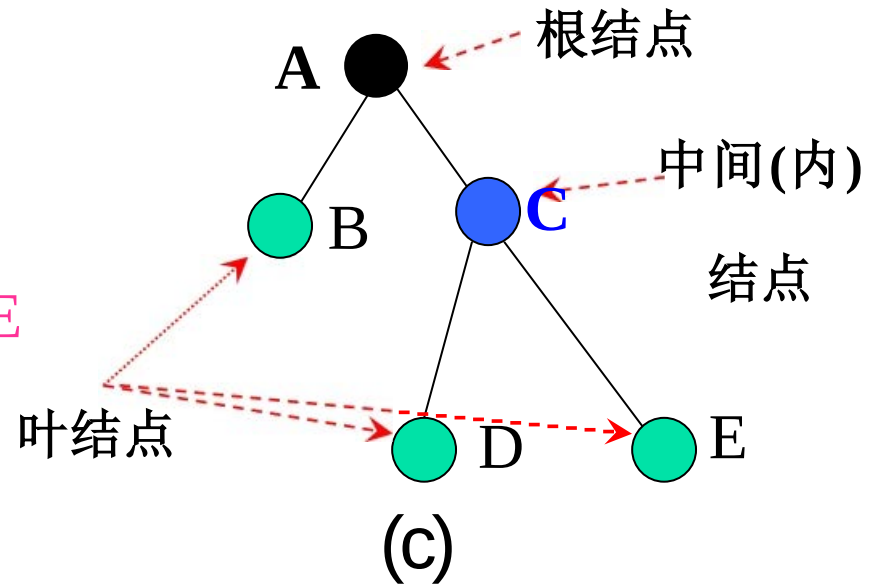
3.1 基本概念



(a)



(b)



(c)

父结点: A 是 B、C 结点的父结点; C 是 D、E 的父结点。

子结点: B、C 是 A 的子结点; D、E 是 C 的子结点。

兄弟结点: B、C 互为兄弟结点; D、E 互为兄弟结点。



3.1 基本概念

□ 字符串 (string)

- 定义：假定 Σ 是字符的有限集合，它的每一个元素称之为字符。由 Σ 中字符相连而成的有限序列被称之为 Σ 上的字符串（或称符号串，或称链）。特殊地，不包括任何字符的字符串称为空串，记作 ε 。
- 符号串的长度：符号串中符号的个数。符号串 x 的长度用 $|x|$ 表示。 $|\varepsilon| = 0$ 。

包括空串的 Σ 上字符串的全体记为 Σ^* 。



3.1 基本概念

□字符串操作

假定 Σ 是字符的有限集合, x, y 是 Σ 上的符号串

(1) 字符串连接: 则把 y 的各个符号写在 x 的符号之后得到的符号串称为 x 与 y 的连接, 记作 xy 。

例如: $\Sigma=\{a, b, c\}, x=ab, y=cba$

那么, $xy=abcba$



3.1 基本概念

- 设 x 是符号串，把 x 自身连接 n 次得到的符号串，即 $z=xx\dots x$ (n 个 x), 称为 x 的 n 次方幂，记作 x_n 。
- 注意: $x_0 = \varepsilon$

$$x_n = xx_{n-1} = x_{n-1}x \quad (n \geq 1)$$

$$x^* = x_n \quad (n \geq 0), \quad x_+ = x_n \quad (n \geq 1)$$

例如：如果 $x=a$, 则 $x_1=a$, $x_2=aa$, $x_3=aaa$,

如果 $x=ab$, 则 $x_0 = \varepsilon$, $x_3=ababab$



3.1 基本概念

(2) 符号串集合的乘积

设A, B是符号串的集合, 则A, B的乘积定义为:

$$AB = \{xy \mid x \in A, y \in B\}$$

相应地, $A_0 = \{\varepsilon\}$, $A_n = A_{n-1}A = AA_{n-1}$

例如: 设 $A = \{aa, bb\}$, $B = \{cc, dd, ee\}$, 则

$$AB = \{aacc, aadd, aaee, bbcc, bbdd, bbee\}$$

$$A_2 = \{aaaa, aabb, bbaa, bbbb\}$$



3.1 基本概念

(3) 闭包

如果 V 是字符表 Σ 上的字符串集合，那么， V 的闭包定义为： $V^* = V_0 \cup V_1 \cup V_2 \cup \dots$

$V_+ = V_1 \cup V_2 \cup \dots$ （称为 V 的正闭包）

$$V_+ = V^* - \{\varepsilon\}$$

例如： $V = \{a, b\}$

$$V^* = \{\varepsilon, a, b, aa, ab, bb, ba, aaa, \dots\}$$

$$V_+ = \{a, b, aa, ab, ba, bb, aaa, \dots\}$$



3.1 基本概念

□正则表达式（简称正则式）：

正则式对应于 Σ 上的一些子集(正则集)，并通过递归定义：

- 1) 空集 ϕ 和空字符串 ε 是正则式，它们的正则集分别为 ϕ 和 $\{\varepsilon\}$ ；
- 2) 任何 $x \in \Sigma$, x 是正则式，它对应的正则集是 $\{x\}$ ；
- 3) 如果 X, Y 是 Σ 上的正则式，并且它们对应的正则集分别为 U, V ，那么， $X|Y$, $X \cdot Y$ 和 X^* 也是正则式，且它们对应的正则集分别为 $U \cup V$, $U \cdot V$ 和 U^* 。



3.1 基本概念

例如： 假设 $\Sigma = \{0, 1\}$ ，那么，0 和 1 都是正则表达式。如果令 $x=0$, $y=1$, 那么，

$y^* = 1^*$ 也是正则式，对应的正则集为：

$$U = \{\varepsilon, 1, 11, \dots\}$$

$xy^* = 01^*$ 也是正则式，且它对应的正则集：

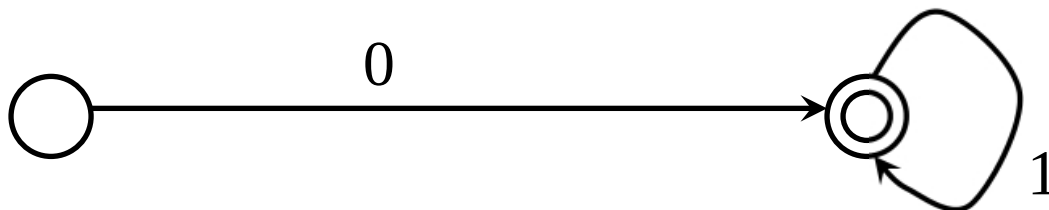
$$V = \{0, 01, 011, 0111, \dots\} x|y^*$$

$$= \{x\} \cup U = \{0, \varepsilon, 1, 11, 111, \dots\}$$

3.1 基本概念

□ 正则表达式与有限状态图

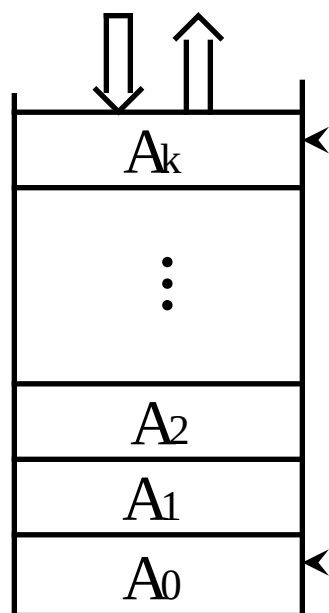
正则表达式可以用有向图表示，图的结点是状态，有一个起始结点和一个终止结点。起始结点只有出边，终止结点用双圆圈表示。边上的符号表示从一个状态到另一个状态结点允许出现的字符，这种图称之为有限状态图。正则式 01^* 对应的有限状态图为：



3.1 基本概念

□ 栈 (stack)

栈是一种线性表， $A = A_0, A_1, \dots, A_k$ 。其中， A_0 是栈底， A_k 是栈顶，当栈为空时， A_0 既是栈顶也是栈底。



Top

Last in, first out

Bottom



3.2 形式语言



3.2 形式语言

□关于语言的定义

人类所特有的用来表达意思、交流思想的工具，是一种特殊的社会现象，由语音、词汇和语法构成一定的系统。

—商务印书馆，《现代汉语词典》，1996

按照一定规律构成的句子和符号串的有限或无限的集合。

— Chomsky

语言可以被看成一个抽象的数学系统。

—吴蔚天，1994



3.2 形式语言

□ 语言描述的三种途径

- ❖ 穷举法 — 只适合句子数目有限的语言。
- ❖ 语法描述 — 生成语言中合格的句子。
- ❖ 自动机 — 对输入的句子进行检验，区别哪些是语言中的句子，哪些不是语言中的句子。



3.2 形式语言

□形式语言的直观意义

形式语言是用来精确地描述语言（包括人工语言和自然语言）及其结构的手段。形式语言学 也称 代数语言学。

以重写规则 $\alpha \rightarrow \beta$ 的形式表示，其中， α ， β 均为字符串。顾名思义：字符串 α 可以被改写成 β 。一个初步的字符串通过不断地运用重写规则，就可以得到另一个字符串。通过选择不同的规则并以不同的顺序来运用这些规则，就可以得到不同的新字符串。



3.2 形式语言

□形式语法的定义

形式语法是一个4元组 $G=(N, \Sigma, P, S)$, 其中 N 是非终结符的有限集合(有时也叫变量集或句法种类集); Σ 是终结符的有限集合, $N \cap \Sigma = \Phi$; $V=N \cup \Sigma$ 称总词汇表; P 是一组重写规则的有限集合: $P=\{ \alpha \rightarrow \beta \}$, 其中, α, β 是 V 中元素构成的串, 但 α 中至少应含有一个非终结符号; $S \in N$, 称为句子符或初始符。

例如: $G = (\{A, S\}, \{0, 1\}, P, S)$

$P: S \rightarrow 0 A 1 \quad 0 A \rightarrow 00A1 \quad A \rightarrow 1$



3.2 形式语言

□ 推导的定义

设 $G=(N, \Sigma, P, S)$ 是一个文法, 在 $(N \cup \Sigma)^*$ 上定义关系 $\Rightarrow_G$ (直接派生或推导) 如下:

如果 $\alpha\beta\gamma$ 是 $(N \cup \Sigma)^*$ 中的符号串, 且 $\beta \rightarrow \delta$ 是 P 的产生式, 那么 $\alpha\beta\gamma \Rightarrow_G \alpha\delta\gamma$ 。



3.2 形式语言

用 $\Rightarrow_G$ (按非平凡方式派生) 表示 $\Rightarrow$ 的传递闭包, 也就是 $(N \cup \Sigma)^*$ 上的符号串 ξ_i 到 ξ_{i+1} 的 n ($n \geq 1$) 步推导或派生。

用 $\overset{*}{\Rightarrow}_G$ (派生) 表示 $\Rightarrow_G$ 的自反和传递闭包, 即由 $(N \cup \Sigma)^*$ 上的符号串 ξ_i 到 ξ_{i+1} 经过 n ($n \geq 0$) 步的推导或派生。

如果清楚某个推导是文法 G 所产生的, 则符号 $\overset{*}{\Rightarrow}_G$ 或 $\overset{+}{\Rightarrow}_G$ 中的 G 可以省略不写。



3.2 形式语言

□最左推导、最右推导和规范推导

约定每步推导中只改写最左边的那个非终结符，这种推导称为“最左推导”。

约定每步推导中只改写最右边的那个非终结符，这种推导称为“最右推导”。

最右推导也称规范推导。

3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T$

3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T * F$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T * F$
 $\Rightarrow a+F * F$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F$
 $\Rightarrow a+F*F \Rightarrow a+a*F$



3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F$
 $\Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a$ (最左推导)

3.2 形式语言

例 3.1: $G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$

$P: E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

字符串 $a+a*a$ 的两种推导过程:

$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F$

$\Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a$ (最左推导)

$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*a \Rightarrow E+F*a \Rightarrow E+a*a$

$\Rightarrow T+a*a \Rightarrow F+a*a \Rightarrow a+a*a$ (最右推导)



3.2 形式语言

□句型与句子

一些特殊类型的符号串为文法 $G=(N, \Sigma, P, S)$ 的句子形式(句型):

- (1) S 是一个句子形式;
- (2) 如果 $\alpha\beta\gamma$ 是一个句子形式, 且 $\beta \rightarrow \delta$ 是 P 的产生式, 则 $\alpha\delta\gamma$ 也是一个句子形式;

文法 G 的不含非终结符的句子形式称为 G 生成的句子。由文法 G 生成的语言, 记作 $L(G)$, 指 G 生成的所有句子的集合。即: $L(G)=\{x \mid x \in \Sigma, S \xRightarrow{*} x\}$



3.2 形式语言

□ 正则文法

如果文法 $G=(N, \Sigma, P, S)$ 的 P 中的规则满足如下形式: $A \rightarrow Bx$, 或 $A \rightarrow x$, 其中 $A, B \in N$, $x \in \Sigma$, 则称该文法为正则文法 或称 3型文法。(左线性正则文法)

如果 $A \rightarrow xB$, 则该文法称为右线性正则文法。

3.2 形式语言

例 3.2: $G = (N, \Sigma, P, S)$,

$N = \{S, A, B\}, \quad \Sigma = \{a, b\},$

$P:$ (a) $S \rightarrow a A$ (b) $A \rightarrow a A$

(c) $A \rightarrow b b B$ (d) $B \rightarrow b B$

(e) $B \rightarrow b$

$A \rightarrow b B'$
 $B' \rightarrow b B'$

$L(G) = \{a^n b^m\}, n \geq 1, m \geq 3$

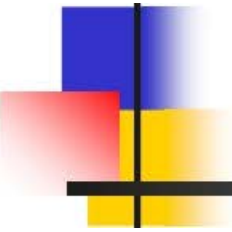


3.2 形式语言

□ 上下文无关文法

(CFG, context-free grammar)

如果 P 中的规则满足如下形式: $A \rightarrow \alpha$, 其中 $A \in N$, $\alpha \in (N \cup \Sigma)^*$, 则称该文法为上下文无关文法 (CFG) 或称 2型文法。



3.2 形式语言

例 3.3: $G = (N, \Sigma, P, S),$

$N = \{S, A, B, C\}, \quad \Sigma = \{a, b, c\},$

$P: (a) S \rightarrow A B C$

$(b) A \rightarrow a A \mid a$

$(c) B \rightarrow b B \mid b$

$(d) C \rightarrow B A \mid c$

$L(G) = \{a^n b^m a^k c^\alpha\}, n \geq 1, m \geq 1, k \geq 0, \alpha \in \{0, 1\}$

(如果 $k > 0$ 的话, $\alpha = 1$, 否则, $\alpha = 0$ 。)



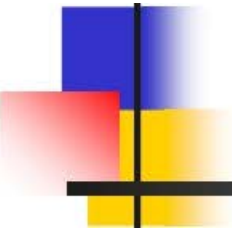
3.2 形式语言

□ 上下文有关文法

(context-sensitive grammar, CSG)

如果 P 中的规则满足如下形式: $\alpha A \beta \rightarrow \alpha \gamma \beta$, 其中 $A \in N$, $\alpha, \beta, \gamma \in (N \cup \Sigma)^*$, 且 γ 至少包含一个字符, 则称该文法为上下文有关文法(CSG)或称 1型文法。

另一种定义: if $x \rightarrow y$, $x \in (N \cup \Sigma)^+$,
 $y \in (N \cup \Sigma)^*$, 并且 $|y| \geq |x|$ 。



3.2 形式语言

例 3.4: $G = (N, \Sigma, P, S)$

$N = \{S, A, B, C\}, \quad \Sigma = \{a, b, c\},$

$P:$ (a) $S \rightarrow A B C$ (b) $A \rightarrow a A \mid a$
(c) $B \rightarrow b B \mid b$ (d) $B C \rightarrow B c c$

$L(G) = \{a^n b^m c^2\}, n \geq 1, m \geq 1$



3.2 形式语言

□ 无约束文法（无限制重写系统）

如果 P 中的规则满足如下形式： $\alpha \rightarrow \beta$ ， α ， β 是字符串，则称 G 为无约束文法，或称 0 型文法。



3.2 形式语言

显然，每一个正则文法都是上下文无关文法，每一个上下文无关文法都是上下文有关文法，而每一个上下文有关文法都是 0 型文法，即：

$$L(G_0) \supseteq L(G_1) \supseteq L(G_2) \supseteq L(G_3)$$

3.2 形式语言

□ 语言与文法类型的约定

如果一种语言能由几种文法所产生，则把这种语言称为在这几种文法中受限制最多的那种文法所产生的语言。

例 3.5: $G = (\{S, A, B\}, \{a, b\}, P, S)$

$P: S \rightarrow aB \quad S \rightarrow bA \quad A \rightarrow aS \quad A \rightarrow bAA$

$A \rightarrow a \quad B \rightarrow bS \quad B \rightarrow aBB \quad B \rightarrow b$

G 为上下文无关文法。

$L(G) = \{ \text{等数量的 } a \text{ 和 } b \text{ 构成的链} \}$



3.2 形式语言

□CFG 产生的语言句子的派生树表示

CFG $G=(N, \Sigma, P, S)$ 产生一个句子的派生树由如下步骤构成:

- (1) 对于 $\forall x \in N \cup \Sigma$ 给一个标记作为节点, S 作为树的根节点。
- (2) 如果一个节点的标记为 A , 并且它至少有一个除它自身以外的后裔, 则 $A \in N$ 。
- (3) 如果一个节点的标记为 A , 它的 $k (k > 0)$ 个直接后裔节点按从左到右的次序依次标记为 $A_1, A_2, \dots, A_k$, 则 $A \rightarrow A_1 A_2 \dots A_k$ 一定是 P 中的一个产生式。

3.2 形式语言

例如, $G = (\{S, A\}, \{a, b\}, P, S)$

$P: S \rightarrow bA \quad A \rightarrow bAA \quad A \rightarrow a$

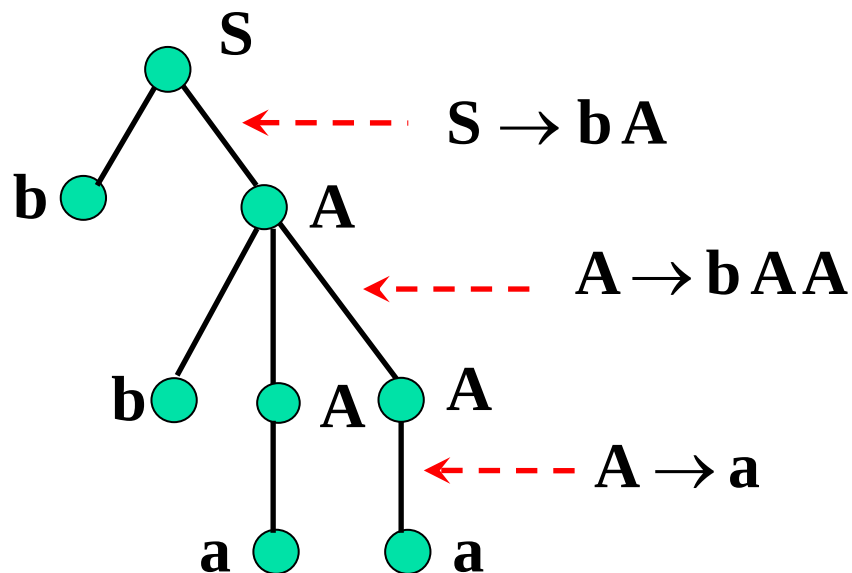
G 所产生的句子 $bbaa$ 可以由下面的生树表示:

$S \Rightarrow bA$

$\Rightarrow bbAA$

$\Rightarrow bbaA$

$\Rightarrow bbaa$





3.2 形式语言

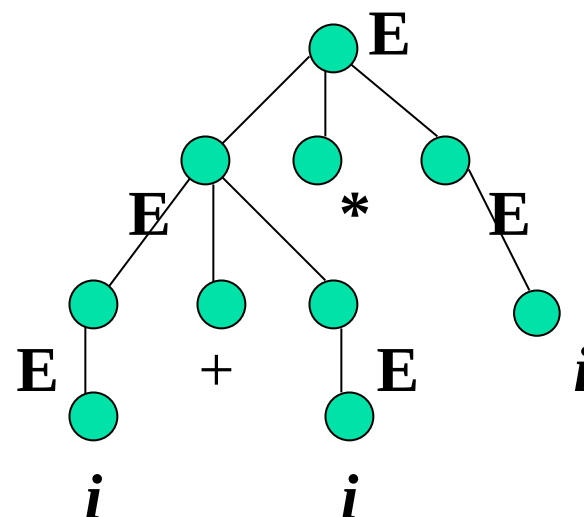
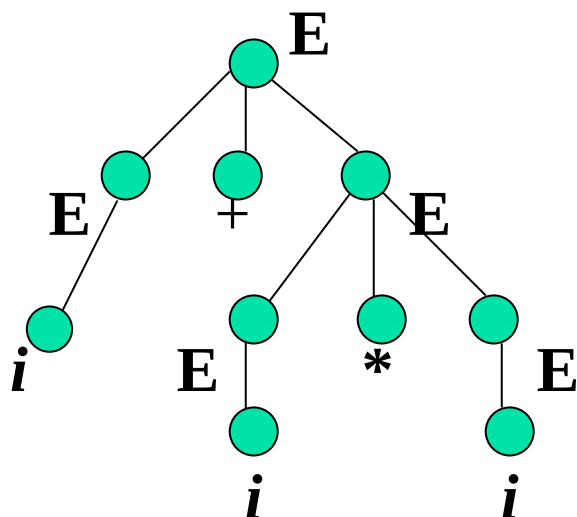
□ 上下文无关文法的二义性

一个文法 G ，如果存在某个句子有不只一棵分析树与之对应，那么称这个文法是二义的。

3.2 形式语言

例: $G(E): E \rightarrow E + E \mid E * E \mid (E) \mid E - E \mid i$

对于句子 $i + i * i$ 有两棵对应的分析树。



3.2 形式语言

例：给定文法 $G(S)$ ：

① $S \rightarrow P NP \mid PP Aux NP$

② $PP \rightarrow P NP$

③ $NP \rightarrow NN \mid NP Aux NP$

④ $P \rightarrow \text{关于}$

⑤ $NN \rightarrow \text{鲁迅} \mid \text{文章}$

⑥ $Aux \rightarrow \text{的}$

短语 “关于鲁迅的文章” 的推导。

$$\begin{aligned} S &\Rightarrow P NP \Rightarrow \text{关于 } NP \Rightarrow \text{关于 } NP Aux NP \\ &\Rightarrow \text{关于 } NN Aux NP \Rightarrow \text{关于鲁迅 } Aux NP \\ &\Rightarrow \text{关于鲁迅的 } NP \Rightarrow \text{关于鲁迅的 } NN \\ &\Rightarrow \text{关于鲁迅的文章} \end{aligned}$$

3.2 形式语言

例：给定文法 $G(S)$ ：

① $S \rightarrow P NP \mid PP Aux NP$

② $PP \rightarrow P NP$

③ $NP \rightarrow NN \mid NP Aux NP$

④ $P \rightarrow \text{关于}$

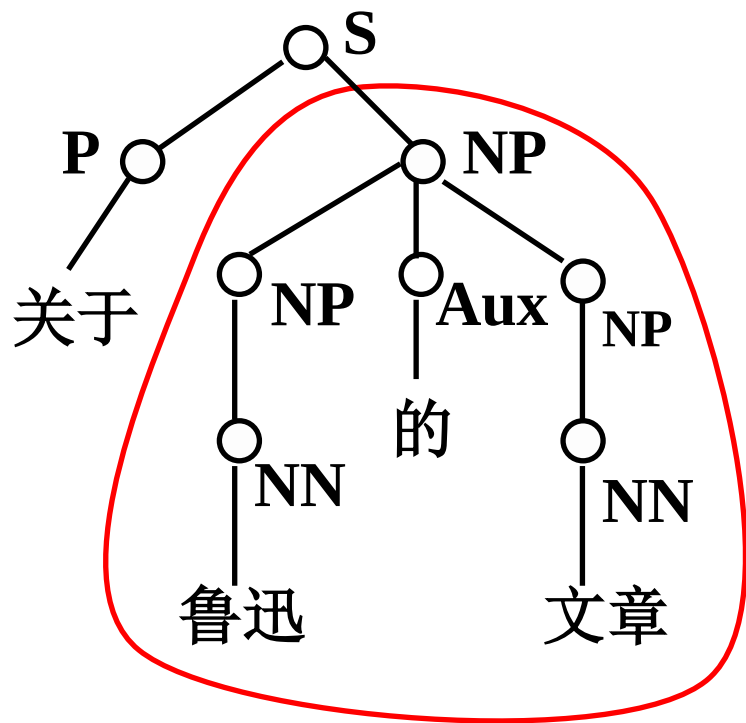
⑤ $NN \rightarrow \text{鲁迅} \mid \text{文章}$

⑥ $Aux \rightarrow \text{的}$

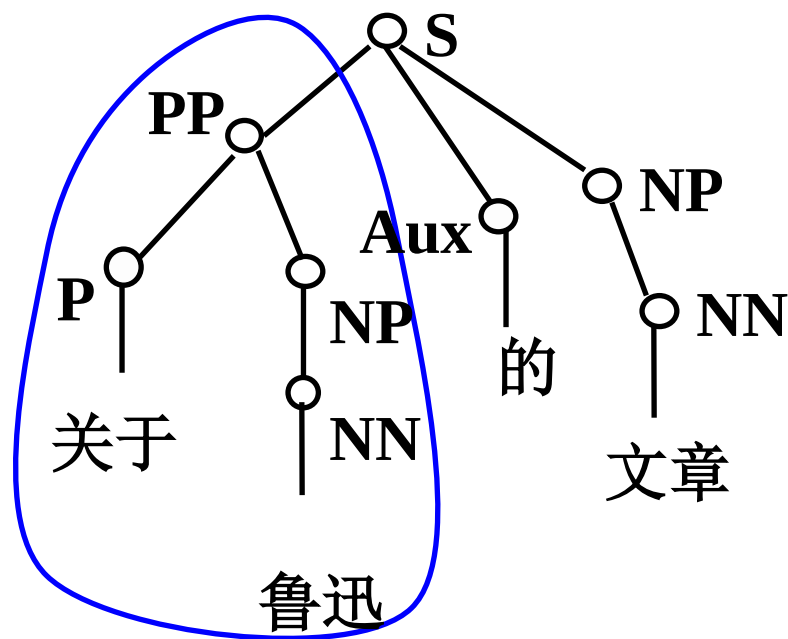
短语 “关于鲁迅的文章” 的推导。

$$\begin{aligned} S &\Rightarrow PP Aux NP \Rightarrow P NP Aux NP \Rightarrow \text{关于 } NP Aux NP \\ &\Rightarrow \text{关于 } NN Aux NP \Rightarrow \text{关于鲁迅 } Aux NP \\ &\Rightarrow \text{关于鲁迅的 } NP \Rightarrow \text{关于鲁迅的 } NN \\ &\Rightarrow \text{关于鲁迅的文章} \end{aligned}$$

3.2 形式语言



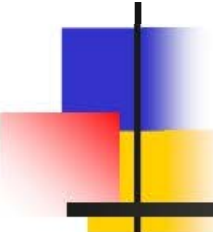
短语的派生树—(1)



短语的派生树—(2)



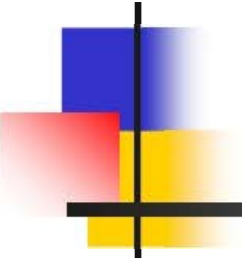
3.3 自动机



语言与识别器的对应关系

识别器是有穷地表示无穷语言的另一种方法。每一个语言的句子都能被一定的识别器所接受。

语言类型	识别器类型
0 型	图灵机
1 型	线性带限自动机
2 型	下推自动机
3 型	有限自动机



3.3.1 有限自动机与正则文法



3.3.1 有限自动机与正则文法

□确定的有限自动机(Definite Automata, DFA)

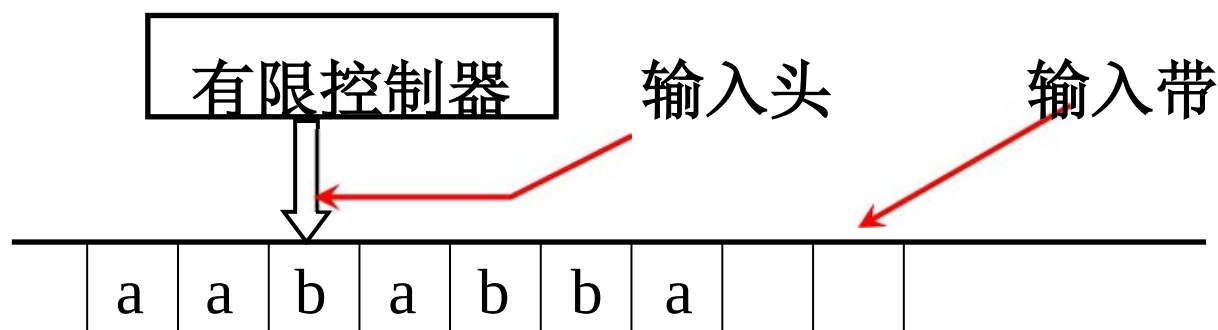
确定的有限自动机 M 是一个五元组:

$$M = (\Sigma, Q, \delta, q_0, F)$$

- Σ 是输入符号的有穷集合;
- Q 是状态的有限集合;
- $q_0 \in Q$ 是初始状态;
- F 是终止状态集合, $F \subseteq Q$;
- δ 是 Q 与 Σ 的直积 $Q \times \Sigma$ 到 Q (下一个状态) 的映射。它支配着有限状态控制的行为, 有时也称为状态转移函数。

3.3.1 有限自动机与正则文法

□ DFA 示意图

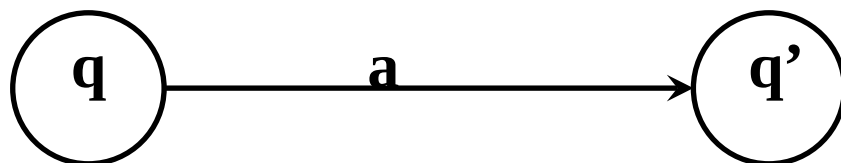


处在状态 $q \in Q$ 中的有限控制器从左到右依次从输入带上读入字符。开始时有限控制器处在状态 q_0 ，并注视 Σ^* 中一个链的最左符号。映射 $\delta(q, a) = q' (q, q' \in Q, a \in \Sigma)$ 表示在状态 q 时，若输入符号为 a ，则自动机进入状态 q' 并且将输入头向右移动一个字符。

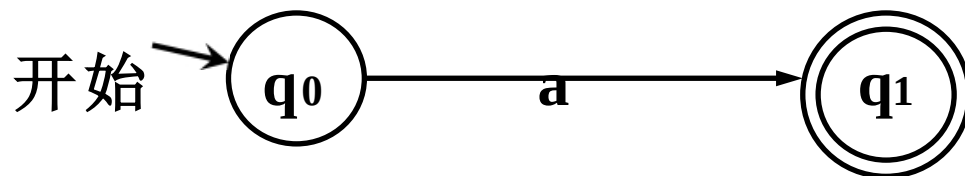
3.3.1 有限自动机与正则文法

□ 状态变换图

映射 $\delta(q, a) = q'$ 可以由状态变换图描述。



为了明确起见，终止状态用双圈表示，起始状态用有“开始”标记的箭头表示。如：





3.3.1 有限自动机与正则文法

□ DFA 定义的语言

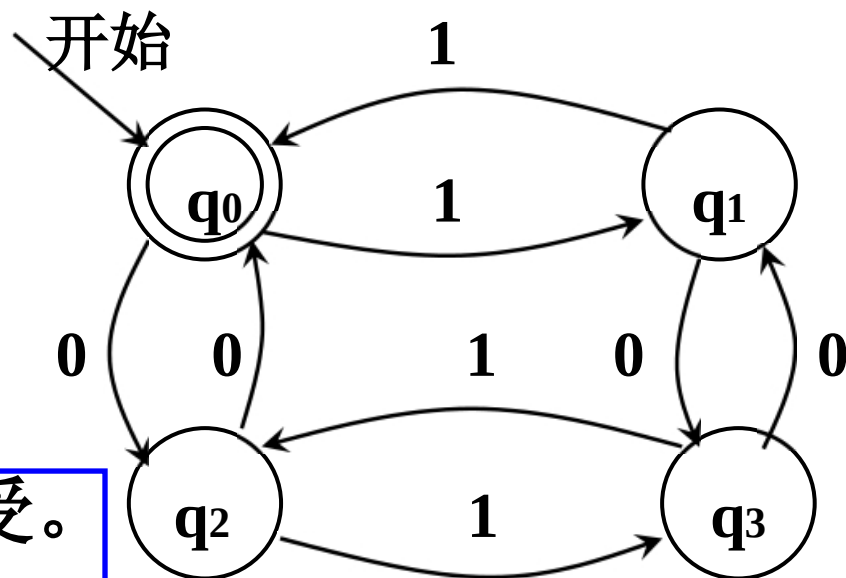
如果一个句子 x 使得有限自动机 M 有 $\delta(q_0, x) = p, p \in F$, 那么, 称句子 x 被 M 接受。由 M 定义的语言 $T(M)$ 就是被 M 接受的句子的全集。即:

$$T(M) = \{ x \mid \delta(q_0, x) \in F \}$$

3.3.1 有限自动机与正则文法

□ DFA 定义的语言

例 3.6



链 $x = 110101$ 被 M 接受。
 $T(M) = \{\text{含偶数个0和偶数个1的链}\}$



3.3.1 有限自动机与正则文法

□ 不确定的有限自动机 (Non-definite Automata, NFA)

不确定的有限自动机 M 是一个五元组

$$M = (\Sigma, Q, \delta, q_0, F)$$

其中, Σ 是输入符号的有穷集合;

Q 是状态的有限集合; $q_0 \in Q$ 是初始状态;

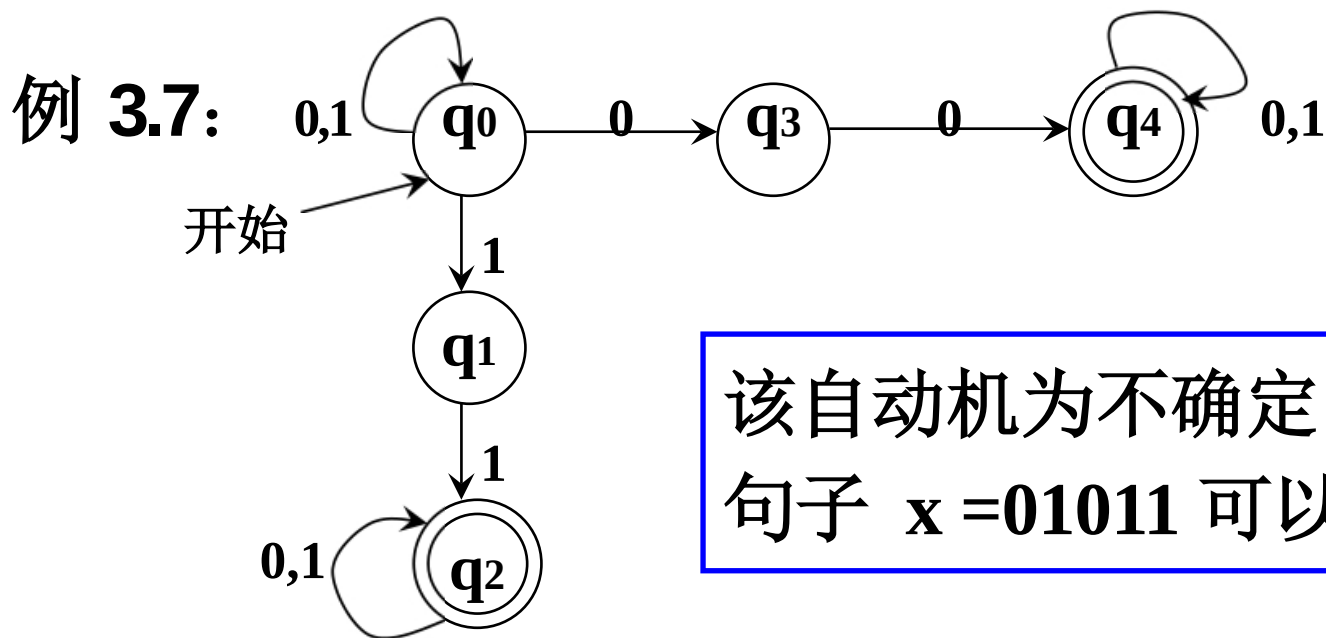
F 是终止状态集合, $F \subseteq Q$;

δ 是 Q 与 Σ 的直积 $Q \times \Sigma$ 到 Q 的幂集 2^Q 的映射。

3.3.1 有限自动机与正则文法

□NFA 与 DFA 的区别

NFA 与 DFA 的唯一区别是：在 NFA 中 $\delta(q, a)$ 是一个状态集合，而在 DFA 中 $\delta(q, a)$ 是一个状态。



该自动机为不确定自动机；
句子 $x = 01011$ 可以被接受。



3.3.1 有限自动机与正则文法

□NFA 与 DFA 的关系

定理 3.1: 设 L 是一个被 NFA 所接受的句子的集合，则存在一个 DFA，它能够接受 L 。

(证明略，数学归纳法)

说明: 由于 DFA 与 NFA 所接受的是同样的链集，所以一般情况下无需区分它们，二者统称为有限自动机 (Finite Automata, FA)。



3.3.1 有限自动机与正则文法

□ 正则文法与有限自动机的关系

定理 3.2: 若 $G = (V_N, V_T, P, S)$ 是一个正则文法, 则存在一个有限自动机 $M = (\Sigma, Q, \delta, q_0, F)$, 使得:
 $T(M) = L(G)$ 。



3.3.1 有限自动机与正则文法

□由 G 构造 M 的一般步骤:

- (1) 令 $\Sigma = V_T$, $Q = V_N \cup \{T\}$, $q_0 = S$, 其中, T 是一个新增加的非终结符。
- (2) 如果在 P 中有产生式 $S \rightarrow \varepsilon$, 则
 $F = \{S, T\}$, 否则 $F = \{T\}$ 。
- (3) 如果在 P 中有产生式 $B \rightarrow a$, $B \in V_N$, $a \in V_T$, 则 $T \in \delta(B, a)$ 。



3.3.1 有限自动机与正则文法

- (4) 如果在 P 中有产生式 $B \rightarrow aC$, $B, C \in V_N$,
 $a \in V_T$, 则 $C \in \delta(B, a)$ 。
- (5) 对于每一个 $a \in V_T$, 有 $\delta(T, a) = \phi$ 。

例 3.8: 给定正则文法 $G = (V_N, V_T, P, S)$, 其中,
 $V_N = \{S, B\}$, $V_T = \{a, b\}$, $P = \{S \rightarrow aB, B \rightarrow bS | aB | a\}$
构造与 G 等价的 NFA。



3.3.1 有限自动机与正则文法

(1) 设 NFA $M=(\Sigma, Q, \delta, q_0, F)$, 根据上述步骤有:

$$\Sigma = V_T = \{a, b\}, \quad Q = V_N \cup \{T\} = \{S, B, T\}, \quad q_0 = S, \\ F = \{T\}$$

(2) 映射 δ 为: $\delta(S, a) = \{B\}$ (因为有规则 $S \rightarrow aB$)

$$\delta(S, b) = \phi$$

$$\delta(B, a) = \{B, T\} \quad (\text{因为有 } B \rightarrow aB, B \rightarrow a)$$

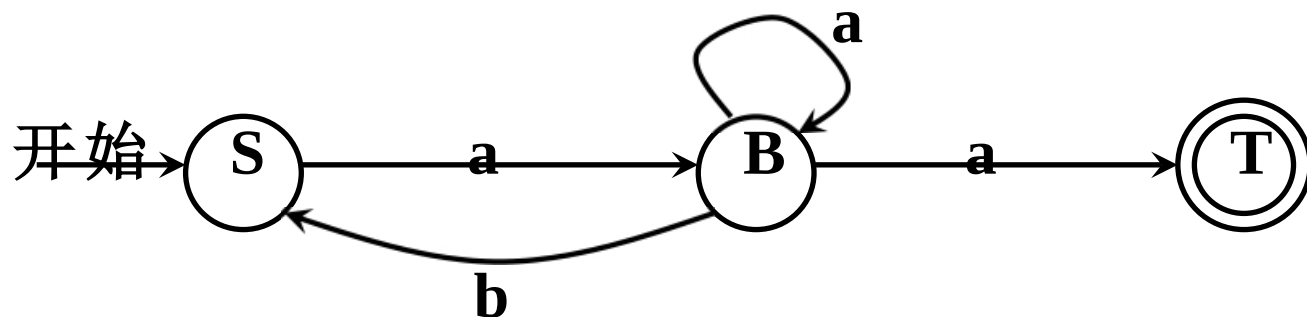
$$\delta(B, b) = \{S\} \quad (\text{因为有 } B \rightarrow bS)$$

$$\delta(T, a) = \phi$$

$$\delta(T, b) = \phi$$

3.3.1 有限自动机与正则文法

等价的 NFA 的状态变换图为：





3.3.1 有限自动机与正则文法

定理 3.2: 若 $M = (\Sigma, Q, \delta, q_0, F)$ 是一有限自动机, 则存在正则文法 $G = (V_N, V_T, P, S)$ 使 $L(G) = T(M)$ 。

由 M 构造 G 的一般步骤:

- (1) 令 $V_N = Q$, $V_T = \Sigma$, $S = q_0$;
- (2) 如果 $C \in \delta(B, a)$, $B, C \in Q$, $a \in \Sigma$, 则在 P 中有产生式 $B \rightarrow aC$;
- (3) 如果 $C \in \delta(B, a)$, $C \in F$, 则在 P 中有产生式 $B \rightarrow a$ 。

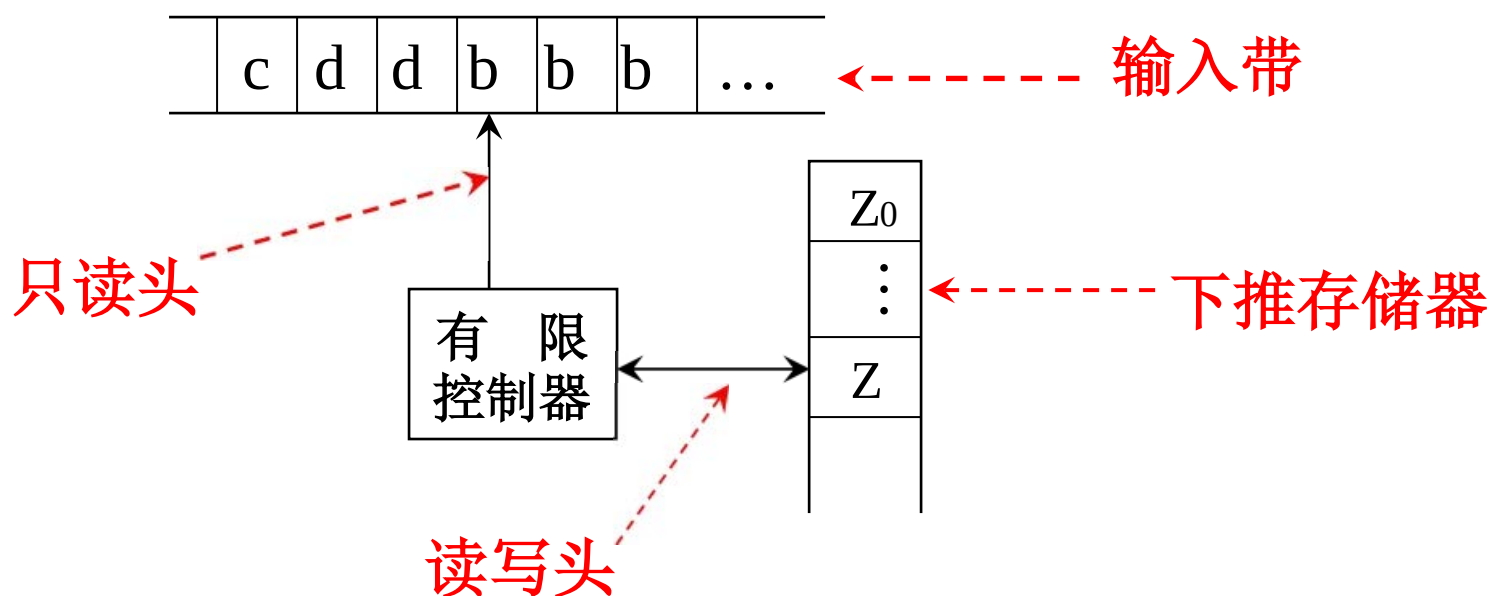


3.3.2 CFG与下推自动机

3.3.2 CFG与下推自动机

□下推自动机 (Push-Down Automata, PDA)

PDA 可以看成是一个带有附加的下推存储器的有限自动机，下推存储器是一个栈。如下图所示：





3.3.2 CFG与下推自动机

□PDA 的定义

一个不确定的PDA可以表达成一个7元组:

$$M = (\Sigma, Q, \Gamma, \delta, q_0, Z_0, F)$$

其中, Σ 是输入符号的有穷集合;

Q 是状态的有限集合; $q_0 \in Q$ 是初始状态;

Γ 为下推存储器符号的有穷集合;

$Z_0 \in \Gamma$ 为下推存储器的起始符号;

F 是终止状态集合, $F \subseteq Q$;

δ 是从 $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ 到 $Q \times \Gamma^*$ 子集的映射。



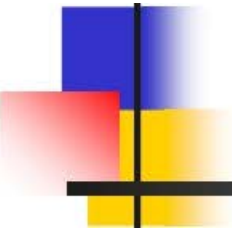
3.3.2 CFG与下推自动机

□映射关系 δ 的解释

映射关系 $\delta(q, a, Z) = \{(q_1, \gamma_1), (q_2, \gamma_2), \dots, (q_m, \gamma_m)\}$

其中, $q_1, q_2, \dots, q_m \in Q, a \in \Sigma, Z \in \Gamma, \gamma_1, \gamma_2, \dots, \gamma_m \in \Gamma^*$ 。

该映射的意思是：当PDA处于状态 q ，面临输入符号 a 时，自动机将进入 $q_i, i = 1, 2, \dots, m$ 状态，并以 γ_i 来代替下推存储器(栈)顶端符号 Z ，同时将输入头指向下一个字符。当 Z 被 γ_i 取代时， γ_i 的符号按照从左到右的顺序依次从下向上推入到存储器。



3.3.2 CFG与下推自动机

特殊情况下，当

$$\delta(q, \varepsilon, Z) = \{(q_1, \gamma_1), (q_2, \gamma_2), \dots, (q_m, \gamma_m)\}$$

时，输入头位置不移动，只用于处理下推存储器内部的操作，叫作 “ ε 移动”。

3.3.2 CFG与下推自动机

□符号约定

设有序对 $(q, \gamma), q \in Q, \gamma \in \Gamma^*$ 对于 $a \in (\Sigma \cup \{\varepsilon\}), \beta \in \Gamma^*, Z \in \Gamma$
如果 $(q', \beta) \in \delta(q, a, Z)$, $q', q \in Q$, 则表达式

$$a: (q, Z\gamma) \mid_M (q', \beta\gamma)$$

表示根据下推自动机的状态变换规则, 输入 a 能使下推自动机 M 由格局 $(q, Z\gamma)$ 变换到格局 $(q', \beta\gamma)$, 或称

$a: (q, Z\gamma) \mid_M (q', \beta\gamma)$ 为合法转移。零次或多次合法转移记为: $a: (q, Z\gamma) \mid_M^* (q', \beta\gamma)$, M 可以省略不写。

3.3.2 CFG与下推自动机

□ 下推自动机接受的语言

下推自动机 M 所接受的语言定义为:

$$T(M) = \{x | x: (q_0, Z_0) \xrightarrow{*}_M (q, \gamma), \gamma \in \Gamma^*, q \in F\}$$

Definition 1: A PDA accepts its input by consuming it and entering an accepting state.

Definition 2: The set of strings that cause the PDA to empty its stack, starting from the initial instantaneous description (ID).

3.3.2 CFG与下推自动机

例 3.9 下推自动机 $M = (\Sigma, Q, \Gamma, \delta, q_0, Z_0, F)$ 接受语言 $L = \{wcw^R \mid w \in \{a, b\}^*\}$, 其中, $Q = \{0, 1\}$,

$\Sigma = \{a, b, c\}$, $\Gamma = \{A, B\}$, $q_0 = 0$, $Z_0 = \#$, $F = \{1\}$, δ 定义如下:

(1) $\delta(0, a, \varepsilon) \vdash \{(0, A)\}$ (2) $\delta(0, b, A) \vdash \{(0, AB)\}$

(3) $\delta(0, b, B) \vdash \{(0, BB)\}$ (4) $\delta(0, c, B) \vdash \{(1, B)\}$

(5) $\delta(1, b, B) \vdash \{(1, \varepsilon)\}$ (6) $\delta(1, a, A) \vdash \{(1, \varepsilon)\}$

3.3.2 CFG与下推自动机

对于输入 abbcbbba 下推自动机 M 的处理步骤为:

状态	输入	栈	运用的规则
0	abbcbbba	#	—
0	bcbba	A#	$\delta(0, a, \varepsilon) \Rightarrow \{(0, A)\}$
0	cbba	BA#	$\delta(0, b, A) \Rightarrow \{(0, AB)\}$
0	bba	BBA#	$\delta(0, b, B) \Rightarrow \{(0, BB)\}$
1	ba	BBA#	$\delta(0, c, B) \Rightarrow \{(1, B)\}$
1	a	BA#	$\delta(1, b, B) \Rightarrow \{(1, \varepsilon)\}$
1		A#	$\delta(1, b, B) \Rightarrow \{(1, \varepsilon)\}$
1	ε	#	$\delta(1, a, A) \Rightarrow \{(1, \varepsilon)\}$

3.3.2 CFG与下推自动机

例 3.6 针对语言 $L = \{a^n b^n : n \geq 0\}$, 我们可以设计一个下推自动机 $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$, 其中

$$Q = \{q_0, q_1, q_2\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{0, 1\},$$

$$z = 0,$$

$$F = q_0.$$

δ 定义如下:

$$\delta(q_0, a, 0) = \{(q_1, 10)\},$$

$$\delta(q_1, a, 1) = \{(q_1, 11)\},$$

$$\delta(q_1, b, 1) = \{(q_2, \varepsilon)\},$$

$$\delta(q_2, b, 1) = \{(q_2, \varepsilon)\},$$

$$\delta(q_2, \varepsilon, 0) = \{(q_0, \varepsilon)\}$$

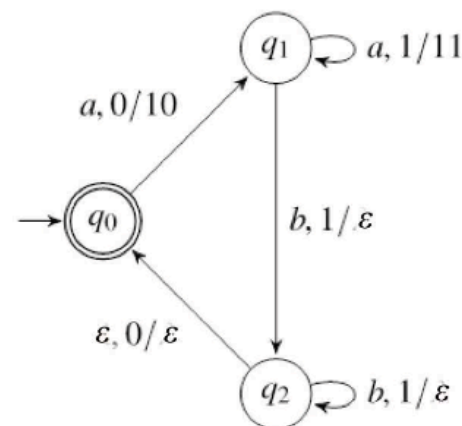


图3.7 识别 $\{a^n b^n : n \geq 0\}$ 的下推自动机

□ 当输入字符串为 **aabb** 时, 自动机 M 的工作过程是: $(q_0, aabb, 0) \vdash (q_1, abb, 10) \vdash (q_1, bb, 110) \vdash (q_2, b, 10) \vdash (q_2, \varepsilon, 0) \vdash (q_0, \varepsilon, \varepsilon)$

3.3.3 下推自动机在NLP中的应用

- 句法分析
- 自然语言的文法
- 句子：“我看见你”
- 句子“我看见你很开心”

$S \rightarrow NP \ VP$	$IP \rightarrow VP$
$NP \rightarrow PN$	$IP \rightarrow NP \ VP$
$NP \rightarrow NP \ PP$	$ADVP \rightarrow AD$
$VP \rightarrow VV \ NP$	$VV \rightarrow \text{看见 吃 做 }\dots$
$VP \rightarrow VV \ IP$	$PN \rightarrow \text{我 你 他 }\dots$
$VP \rightarrow VV \ NP \ IP$	$VA \rightarrow \text{开心 快乐 生气 }\dots$
$VP \rightarrow VA$	$AD \rightarrow \text{很 特别 }\dots$
$VP \rightarrow ADVP \ VP$	

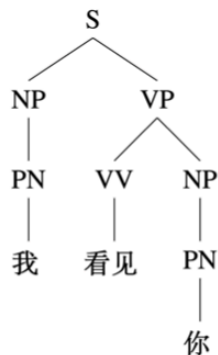


图3.8 “我看见你”的句法分析树

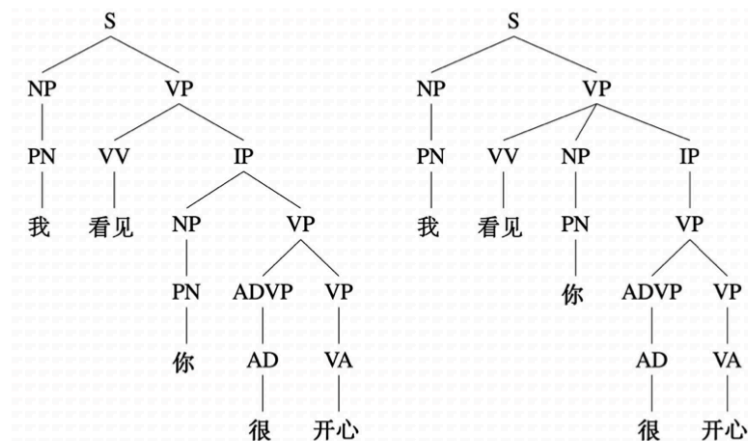


图3.9 “我看见你很开心”的句法分析树



本章小结

- 语言，形式语言
- 乔姆斯基文法体系
- 正则表达式，有穷状态自动机
- 上下文无关文法，下推自动机